Prérequis

Avoir terminé le laboratoire précédent. N'oubliez pas d'importer la base de données fournie.

Qu'est-ce qu'une "session" ?

Le support des sessions de PHP[1] est un moyen de préserver des données entre plusieurs accès (du même ordinateur et du même navigateur) à une application web. Par exemple, retenir quel utilisateur est connecté.

Chaque visiteur qui accède à votre application web se voit assigner un identifiant unique (identifiant de session) qui permet d'identifier sa session. Il y a donc une session par visiteur.

PHP propose un tableau superglobal `$_SESSION` qui permet de stocker les données entre les différentes requêtes du même visiteur.

Pour que cela fonctionne, il faut que chaque page de votre application appelle explicitement la fonction `session_start()` avant d'accéder à ce tableau `$_SESSION` : s'il existe déjà une session pour l'utilisateur actuel, l'environnement précédemment sauvé sera recrée. Dans le cas contraire, une nouvelle session est créée.

---

1. Source : http://php.net/manual/fr/intro.session.php ↩

Utiliser la session lors de la connexion

Nous allons maintenant utiliser les sessions pour retenir le pseudo de l'utilisateur connecté et donc lui proposer du contenu personnalisé.

Indiquer à PHP qu'on veut gérer une session

Tout d'abord, ajoutez la ligne `session_start()` au tout début de votre fichier `functions.php` : comme cette page sera insérée dans toutes les autres pages, les sessions seront gérées pour toutes ces pages.

Au login stocker l'utilisateur dans la session

Ensuite, dans la page `login.php`, si le mot de passe est correct, sauvegardez le pseudo de l'utilisateur fraichement connecté à la clé `user` du tableau `$_SESSION` juste avant de le rediriger vers son profil :

```
if ($row['password'] == $password)
{
    session_regenerate_id(true);
    $_SESSION["user"] = $pseudo;
    redirect("profile.php?pseudo=$pseudo"); // le paramètre GET 'pseudo'
                                            // ne sera bientôt plus nécessaire
ici
}
```

On mémorise la **clé primaire** (ici le pseudo), pas toute la ligne du membre ni le mot de passe. Dans un projet ultérieur, la clé pourra être un `id` alors que le login sera un email.

Renouveler l'identifiant de session à l'authentification

Appelez `session_regenerate_id(true)` juste avant d'écrire dans `$_SESSION` après un login réussi. Cela attribue un **nouvel** identifiant de session et détruit l'ancien (`true`).

Sans cela, une attaque de *fixation de session* est possible : un attaquant impose à la victime un identifiant qu'il connaît *avant* la connexion ; une fois l'utilisateur authentifié, cet identifiant donnerait accès au compte. En renouvelant l'ID au moment du login, l'identifiant « planté » devient inutilisable.

Faites de même à l'**inscription** lorsque vous connectez automatiquement le nouvel utilisateur (écriture de `$_SESSION["user"]`).

Ainsi, si la variable `$_SESSION["user"]` existe, c'est qu'un utilisateur est connecté.

Interdire l'accès aux pages si on n'est pas connecté

Nous allons maintenant faire en sorte que les pages *privées* de l'application ne soient accessibles que si l'utilisateur est connecté. Dans le cas contraire, on le redirigera vers l'index.

Les pages **publiques** (accessibles sans être connecté) sont : `index.php` , `login.php` et `signup.php` (l'inscription sera vue juste après). Les pages `friends.php` et `messages.php` restent pour l'instant des *stubs* incomplets : on ne leur ajoute pas encore de contrôle d'accès ; elles seront complétées (et sécurisées) dans la leçon 04.

Comme nous allons devoir réaliser ce comportement à plusieurs endroits, créons une fonction (dans le fichier `functions.php`) qui s'en occupe :

```
function check_login()
{
    global $user;
    if (!isset($_SESSION['user']))
        redirect('index.php');
    else
        $user = $_SESSION['user'];
}
```

Remarquez que cette fonction crée ou modifie la variable *globale* `$user` . Cette variable sera donc accessible après l'appel à cette fonction.

> **Remarque**
>
> Si l'utilisateur n'est pas connecté, il est redirigé vers la page `index.php`. Après l'appel à cette fonction, si le script continue (et que donc il n'y a pas eu de redirection), la variable globale `$user` a donc été créée et est disponible.

Appelez cette fonction (assez tôt) dans les fichiers `profile.php` et `members.php`. Remarquez que l'appel à `members.php` n'affiche la liste des membres que si vous êtes connectés. Vous l'appellerez aussi dans les pages privées que vous compléterez ensuite (`edit_profile.php`, etc.).

Permettre la déconnexion

Nous allons maintenant nous occuper de la déconnexion des utilisateurs. Pour cela, créez un fichier `logout.php`. L'appel à cette ressource devra supprimer les données stockées dans le tableau de session et détruire la session de la façon suivante :

```php
<?php
    require_once("functions.php");
    $_SESSION = [];    //on écrase le tableau
    session_destroy();      //on détruit la session
    redirect('index.php');  //le visiteur est redirigé à l'index
?>
```

Remarquez que dans le menu, il existe déjà un bouton *Log Out* lié à `logout.php` : essayez le.

Afficher le profil de l'utilisateur connecté par défaut

Modifiez la page `profile.php` afin qu'elle affiche le profil de l'utilisateur connecté si on l'appelle sans lui donner de paramètre *GET*. Ainsi, l'appel à http://localhost/my_social_network_base/profile.php affichera le profil du visiteur connecté au lieu d'afficher une erreur :

```php
require_once "functions.php";

check_login();

if(isset($_GET["pseudo"])){
    $pseudo = trim($_GET["pseudo"]);
}
else {
    $pseudo = $user;
}
```

Dans le menu de `profile.php`, le lien *Home* peut à nouveau pointer simplement vers `profile.php` (sans `?pseudo=…`) : la session suffit désormais.

Enfin, modifiez la page `login.php` afin qu'elle redirige l'utilisateur correctement connecté vers cette ressource sans paramètre.

Formulaire d'inscription

Attention

Cet exercice est beaucoup moins dirigé que les précédents, à vous d'écrire le code nécessaire pour le réaliser.

Vous allez maintenant compléter le formulaire d'inscription (`signup.php`). Ce formulaire envoie ses données via la méthode *POST* à la même ressource, `signup.php` . Rappel : cette page reste **publique** — n'appelez pas `check_login()` dessus (un visiteur non connecté doit pouvoir s'inscrire).

Avant d'ajouter un utilisateur dans la base de données, vous devez vérifier les entrées de l'utilisateur. Après un `trim` sur chaque champ POST (sans échappement HTML — la donnée stockée reste brute), vérifiez les règles suivantes :

- Le nom d'utilisateur est obligatoire (pour simplifier, le mot de passe n'est pas obligatoire).
- Le nom d'utilisateur n'existe pas encore dans la base de données
- La taille du nom d'utilisateur est minimum 3 caractères
- Les deux mots de passe entrés doivent être identiques

Si toutes ces conditions ne sont pas vérifiées, n'insérez rien dans la base de données et créez un tableau de *strings* contenant un ou plusieurs messages d'erreurs. Les messages de ce tableau seront affichés en dessous du formulaire, dans une liste à points englobée dans un `<div class="errors">` . Veillez à ce que ce formulaire soit pré-rempli avec les données précédemment encodées par l'utilisateur (utilisez `escape` dans les attributs `value`).

Si les conditions sont vérifiées, vous pouvez insérer le nouvel utilisateur dans la base de données, le connecter via une variable de session et le rediriger vers son profil. Comme pour le login, appelez `session_regenerate_id(true)` juste avant d'écrire `$_SESSION["user"]` (voir l'encart sur la fixation de session dans la page précédente). Pour rappel, voici un exemple de requête SQL *INSERT* :

```
INSERT INTO Members(pseudo, password)
VALUES ("boris", "1234");
```

Modification du profil

Vous allez maintenant vous occuper de la page `edit_profile.php` qui permet de modifier le profil de l'utilisateur courant. L'*upload* d'image de profil est prévu pour le labo suivant (03d) : dans ce labo, vous ne modifiez que la description.

1. Tout d'abord, arrangez-vous pour que cette page ne soit accessible qu'à un utilisateur identifié. Faites également en sorte que sa description apparaisse dans le champ prévu à cet effet et que son image, si elle existe, soit affichée dans l'élément `<img>` du formulaire (rappelez-vous : `escape` à l'affichage / dans le `<textarea>`).
2. Ensuite, complétez le code afin que l'utilisateur puisse modifier sa description, sans vous soucier de son image. En entrée, appliquez `trim` sur le champ POST (donnée brute en base). Pour rappel, voici un exemple de requête SQL *UPDATE* :

   ```
   UPDATE Members
   SET profile = "blablabla", picture_path="blabla.png"
   WHERE pseudo = "blabla";
   ```

   Arrangez vous également pour afficher un message de succès dans un `<div class="success">` après le formulaire.

Pour fixer les idées, voici cette page après une mise à jour de la description :

Update Your Profile

Home | Members | Friends | Messages | Edit Profile | Log Out

Enter or edit your details and/or upload an image.

Je suis Benoît...

Image:

Choose File No file chosen



Save Profile

Your profile has been successfully updated.